



Compute, Routing, and Proof

Zach Kelling

Lux Industries

2026

Contents

1	The position today	4
1.1	The technical spine	4
2	The deployed system	5
3	What the design gets right	6
3.1	The attestation stack	6
3.2	The bounty	6
3.3	Treasury bifurcation, and the July 2026 incident	6
3.4	Receipt-derived accounting, and the delegation primitive	6
4	The unit of account	7
4.1	Revenue is a function of time; cost is a function of choice	7
4.2	Routing alpha, defined	7
4.3	Price discovery, not a trading strategy	7
4.4	The same functional at a lower floor	8
4.5	Committing to an admissible set	8
4.6	Why disclosure needs verification	9
4.7	Why verification wants an execution environment you specify	10
4.8	The chain, restated	10
5	Confidentiality	11
5.1	The asymmetry in the transport	11
5.2	What survives the obvious fix	11
5.3	Peer identity is asserted, not resolved	11
5.4	A router that learns from traffic it cannot read	12
5.5	Custody follows confidentiality	12
6	Two repairs available with deployed primitives	13
6.1	Delegated sessions	13
6.2	Anchoring the compute domain to the capital domain	14
7	Sessions as an agent substrate	15
8	What we bring, named to repositories	16
9	The outcome	16
9.1	The funnel	16
9.2	What the usage is worth	17
9.3	Sequence, and why us	17
10	Sequencing	17
10.1	What blocks what	17
10.2	What could ship in a quarter	18
10.3	What is a year	18
10.4	Staffing, realistically	19
11	Limits	19
11.1	What no one in this programme can fix	19
11.2	What depends on data the network does not publish	19
11.3	Where our interests and yours diverge	19

1 The position today

Morpheus emits about 12,000 MOR a day, down from 14,400 at launch [verified]. Real protocol revenue, the Lido and Aave yield on staked deposits, is about \$476K a year and falling; at the 2024 launch it was \$3.24M a quarter [verified]. Emission outruns real revenue by roughly ten to one [verified]. MOR trades at \$1.58, a market capitalization near \$14M, against on-chain DEX depth of about \$1.3M that is mostly protocol-owned and held in a single Safe, on 24-hour volume near \$57K [verified]. Seventy-five percent of the yield funds a buyback of about \$511K a year [verified].

This is not a Ponzi. Principal is withdrawable one to one, and the capital pool holds about \$20M of third-party stETH [verified]. The problem is narrower and worse for a token. Two facts set the whole argument.

First, revenue is decoupled from usage. It is yield on deposits, so it scales with capital and falls with rates, not with inference sold. The one number that would justify the emission, paid inference, is published nowhere in the 62 repositories [verified]. Eighteen providers are listed on `active.mor.org`, largely reselling Venice and OpenAI subscriptions [verified]. No usage, session, or revenue figure exists anywhere in the tree [verified].

Second, settlement is fakeable. A provider signs a receipt over seven fields including `inputTokens` and `outputTokens` (`abi.go:12-19`) [verified]. The contract decodes five of them and ignores both token counts and the model (`SessionRouter.sol:236`) [verified]. Payment is `pricePerSecond` times elapsed time. Faked work and a substituted model settle identically to honest work. There is no slashing, the `grep` is empty [verified]; no timelock [verified]; and the token has no mint cap (`MOROFT.sol:61`) [verified]. The owner is an anonymous 5-of-9 [verified].

We are Hanzo AI and Lux Industries. We bring the two things the tree is missing: demand into the market, and a settlement receipt that cannot be faked. Hanzo operates an OpenAI-compatible inference endpoint and a learned router that selects a model per request in microseconds on CPU. Lux operates a blockchain with post-quantum key-encapsulation exposed as EVM precompiles on mainnet. This document states Morpheus's problem in Morpheus's numbers, names each fix to a repository that already exists, and gives one quantified outcome with its assumptions on the table.

Provenance. Every number carries a label. [verified] is read from chain, a repository, or De-fiLlama. [illustrative] is modeled from an assumption stated beside it. [pending] awaits an external approval. [client] is our own reach. Unlabeled code citations are [verified] against the working trees at `MorpheusAIs/*` as of August 2026.

1.1 The technical spine

The economic problem has a technical cause, and it runs in one line. Settlement pays for time, not tokens. If revenue is fixed at bid time and cost of service is unobserved, the choice of which model answers a request is the entire margin. That gap is an arbitrage. Someone will take it. Taking it in the open requires a provider to declare what it may serve; a declaration is only a statement until the network can check that the advertised model answered; checking per request rather than per audit needs verification priced in gas; and no rented chain hosts a precompile its operator did not deploy. Pricing, routing, disclosure, verification, and settlement are one chain of implications, not five topics. The outcome in Section 9 is what closing that chain is worth.

A second chain runs alongside and depends on none of the first. Consumer prompts cross the wire in cleartext while provider responses are encrypted, so the half of the exchange that usually carries the sensitive material is the unprotected half. A captured prompt does not expire, which puts confidentiality on an unbounded horizon and constrains where a provider's keys may live. A third chain is shorter and needs nothing not already deployed: the hosted gateway funds every Web2 caller from one wallet though `openSession` takes the owner as its first argument, and the capital and compute domains each hold their own copy of the emission schedule with nothing checking that they agree.

We have read the Morpheus tree carefully. Several parts of it are better than their reputation, and Section 3 credits them. We have a commercial interest in this market working, and Section 11

states where that interest and yours diverge, because a document that does not say so is not worth reading. Ten one-page dockets accompany this paper, LP-312 through LP-321, each self-contained. This document is the argument that connects them.

2 The deployed system

The facts in this section are read from the working tree at `MorpheusAIs/*` as of August 2026 and stated without comment; the comment starts in Section 3.

Assets and domains. MOR is an ERC-20 on Ethereum, bridged to Arbitrum and Base as a LayerZero OFT. Morpheus operates no chain of its own. Capital contracts live on Ethereum, where `Distribution` splits each period’s emission across deposit pools. The compute market is a single EIP-2535 diamond on Base mainnet at `0x6aBE1d282f72B474E54527D93b979A4f64d3030a`, owned by a 5-of-9 Gnosis Safe at `0x1FE04BC15Cf2c5A2d41a0b3a96725596676eBa1E`, with `SessionRouter`, `Marketplace`, `ModelRegistry`, `ProviderRegistry` and `Delegation` facets.

The bid. A bid is six fields (`IBidStorage.sol:14-21`):

```
struct Bid {
    address provider;
    bytes32 modelId;
    uint256 pricePerSecond;
    uint256 nonce;
    uint128 createdAt;
    uint128 deletedAt;
}
```

One of the six is economic. Bidding is permissionless inside a `bidMinPricePerSecond` / `bidMaxPricePerSecond` band and costs a `bidFee` (`Marketplace.sol:63-90`).

The session. `openSession(address user_, uint256 amount_, bool isDirectPaymentFromUser_, bytes approvalEncoded_, bytes signature_)` validates the caller against the owner under `DELEGATION_RULES_SESSION` (`SessionRouter.sol:90`), pulls `amount_` of MOR from `user_` (:104), and computes `endsAt` from the stipend the stake purchases divided by the bid price. Session identity is `keccak256(user||provider||bidId||nonce)` (:159). Provider revenue on close is $(\text{sessionEnd} - \text{openedAt}) \times \text{pricePerSecond}$, where `sessionEnd` is `min(closedAt, endsAt)` (`_getProviderRewards`, :264). `MIN_SESSION_DURATION` is five minutes and `SIGNATURE_TTL` is ten (`SessionStorage.sol:34-35`).

The stipend. `stakeToStipend(amount, timestamp)` returns $\text{amount} \times \text{getComputeBalance}(t) / (100 \times \text{totalMORSupply}(t))$ (:183). `getComputeBalance` reads the diamond’s own `Pool[]` array at `COMPUTE_POOL_INDEX = 3` and subtracts `providersTotalClaimed`, the cumulative amount providers have drawn (:484). That is the only place realised compute demand feeds back into how much session time the market can sell.

Payment. Neither Base payout is a mint. `_claimForProvider` clamps the claim to `provider.stake - provider.limitPeriodEarned` and then executes `safeTransferFrom(fundingAccount, provider, amount)` (:383–397). On the builder side, `BuilderSubnets.sol:335` does `safeTransferFrom(treasury, staker, toClaim_)`. Payment therefore depends on a balance and an approval held by accounts outside the contract.

The node. `proxy-router` is a Go daemon. :3333 carries `morrpc`, the peer-to-peer protocol between consumer and provider nodes; :8082 is the local administrative API and must not be exposed. A provider’s backend is configuration: `ApiAdapterFactory` is constructed from `modelConfig.ApiType`, `ApiURL` and `ApiKey` (`ai_engine.go:64`), and the OpenAI adapter POSTs to whatever base URL it was given with a bearer token, overwriting the requested model name with the configured one (`openai.go:58,64,70`).

The hosted path. Morpheus-Marketplace-API is a FastAPI service at `api.mor.org` presenting an OpenAI-compatible surface, so Web2 callers never hold a wallet. It maintains its own session pool, its own credit ledger, and its own wallet-link table for users who have proved an address by SIWE.

Governance. MRC authorises the allocation of emission to the compute pool. Its repository's last commit is dated 2025-11-17.

3 What the design gets right

A document that only enumerates gaps invites the reader to discount all of it, and would deserve to be discounted. Four things in this tree are better than what we usually find, and two of them are better than what we have built ourselves.

3.1 The attestation stack

`proxy-router/internal/attestation` is 5,439 lines of Go and it is careful work. The TDX `report_data` is checked against the SHA-256 of the SubjectPublicKeyInfo DER of the TLS certificate belonging to the peer the verifier actually reached, which is the check that stops a valid quote from a genuine enclave being replayed in front of a different endpoint (`verifier.go:292`). The NVIDIA `eat_nonce` is cross-checked against the CPU quote and compared in constant time. Golden measurements are pinned to a Sigstore identity, an OIDC issuer and an identity regexp naming the repository, rather than to a hash in a config file (`golden.go:23-30`).

Two dependencies deserve to be named because they are load-bearing. `VerifyQuote` POSTs the quote to a third party's parsing endpoint and infers validity from the JSON that comes back (`verifier.go:563`); and the measurement registry is fetched unsigned from a CDN copy of an npm package, because the upstream repository was removed (`artifacts_registry.go:16-20`). The comment in that file is honest about why. The engineering is sound and the root of trust underneath it is rented.

3.2 The bounty

The bug bounty is scoped by deployed contract address and requires a proof of concept. That is a materially better design than a scope defined by repository name, because it makes the deployed artefact the object of the claim rather than the source tree, and because a mandatory PoC filters the class of report that consumes reviewer time without producing information. We intend to submit against it rather than around it, including for the transport finding in Section 5.3.

3.3 Treasury bifurcation, and the July 2026 incident

On 2026-07-31 a stipend recycling defect was patched on Base mainnet. `_rewardUserAfterClose` had refunded a consumer's full stake on a late close without anchoring the day-lock to when the session stopped consuming compute, so the same MOR could be recycled inside a UTC day to draw repeated stipends. The current code anchors on `min(closedAt, endsAt)` and locks until the start of the following day, with the reason in a comment naming the pull request.

The part worth crediting is not the fix, it is the containment. Because provider payment runs `safeTransferFrom` from a funding account rather than minting, and because the compute pool is bifurcated from the rest of the emission, the exploitable ceiling was bounded at roughly the compute treasury's daily draw, around one percent of the pool [verified], rather than at the token supply. A design where the payment path cannot mint converted an unbounded economic bug into a bounded one. That property was chosen before the bug existed.

3.4 Receipt-derived accounting, and the delegation primitive

Two smaller things. First, the hosted gateway derives day-locks from close receipts read back from chain (`session_routing_service.py:891`) rather than from its own optimistic bookkeeping, which is the correct direction of trust for a component that cannot correct the chain. Second, `_isValidProviderReceipt` already accepts a receipt signed by the provider, by a delegate holding `DELEGATION_RULES_PROVIDER`, or by `owner()` when the provider address is a contract

(`SessionRouter.sol:537-559`). A keyless provider is already a supported shape. Section 5.5 finishes it rather than inventing it.

We also note the capacity manager interface is factored correctly. `HasCapacity(modelID string) bool` is the whole thing (`capacity_manager.go:12`), with implementations selected by config (:112). Adding admission policy is a case in a switch, not a refactor.

4 The unit of account

4.1 Revenue is a function of time; cost is a function of choice

Let p be a bid's `pricePerSecond`, T the settled session duration, π a provider's policy for choosing which backend answers each request, $c(\pi)$ the expected backend cost per paid second under that policy, and $Q(\pi)$ the delivered quality. Settlement pays pT . It does not read π , cannot read c , and has no representation of Q . Provider profit is

$$\Pi(\pi) = (p - c(\pi)) T,$$

which is maximised by minimising $c(\pi)$ subject to whatever floor the provider chooses to honour. Nothing in the protocol supplies that floor.

This is not a criticism of per-second pricing. Per-token pricing has its own pathologies, needs a tokeniser both sides agree on, and would have to be metered by an oracle the network does not have. Per-second pricing is metered by `block.timestamp`, which is the cheapest honest meter available on a chain you do not operate. The point is narrower: the decision fixes where the margin lives, and the margin lives entirely in model selection.

4.2 Routing alpha, defined

Define the gap between pinning one model and routing optimally at the same quality floor $\bar{q}$:

$$\alpha(\bar{q}) = c(\pi_{\text{pin}}) - \min_{\pi: Q(\pi) \geq \bar{q}} c(\pi).$$

α is bounded below by the mass of requests that a cheaper admissible backend answers at or above the floor, multiplied by the price spread of the admissible pool. It is non-negative by construction, since pinning is feasible. It is non-increasing in $\bar{q}$, which is the fact Section 4.4 turns on.

A learned router computes exactly the constrained minimisation on the right. Hanzo's router, Enso, selects per request in microseconds on CPU, maximising $Q(\pi) - \lambda c(\pi) - \mu \ell(\pi)$ over the set of backends that satisfy the request's service objective, where ℓ is latency. On a per-second market the multipliers are not free parameters. Normalising profit by pT leaves $1 - c(\pi)/p$, so the cost weight scales as $\lambda \propto 1/p$: a cheap bid should route harder. The latency weight has no source in settlement at all. A slower backend serves fewer requests inside the same paid window and therefore costs the provider less, so on this market μ must come from the consumer's own scoring, not from the contract.

An illustrative magnitude, and only that. The network publishes no price, session-length or volume series in any of its 62 repositories, so what follows is shape rather than measurement. Suppose the cheapest admissible backend costs 0.2 against a pinned model's 1.0, and seven requests in ten clear the floor on it. Routed cost is $0.7(0.2) + 0.3(1.0) = 0.44$, so $\alpha = 0.56$ of pinned cost [illustrative]. If pinned service consumed sixty percent of bid revenue, routing takes cost of service to twenty-six percent of revenue [illustrative]. Those inputs are assumptions. The first useful thing anyone can do here is measure them, which is what Section 10 proposes as a first step.

4.3 Price discovery, not a trading strategy

An arbitrage this size does not stay with whoever finds it, and we are not proposing that it should. `postModelBid` is permissionless inside the price band, so as routed providers enter, p is competed down toward $c(\pi^*)$ and the surplus moves to consumers, who buy duration and buy more of it as p falls. Morpheus has no process today that discovers what a session-second is worth. Routing is one.

The constraint that binds first is deliberate and is yours: `_claimForProvider` clamps a period's claims to `provider.stake - provider.limitPeriodEarned` (`SessionRouter.sol:383`). Revenue is bounded by stake, so the arbitrage runs out of capital before it runs out of competition. That is a good property and Section 4.6 depends on it.

4.4 The same functional at a lower floor

$\alpha(\bar{q})$ is non-increasing in $\bar{q}$. A router that quietly serves below its advertised floor earns strictly more than one that honours it, and the difference is the fraud surplus. Routing alpha and substitution fraud are the same functional evaluated at two different floors.

On chain they are indistinguishable, because the backend is configuration and not code. A provider that changes one JSON field to point at a smaller model keeps the entire cost difference, produces no on-chain inconsistency, and requires no cooperation from anyone. The node's own documentation is candid about the adjacent case: `providers/resale/overview.mdx` instructs a resale provider to bid on somebody else's model without the `tee` tag, on the grounds that it does not control the backend. That is honest guidance. It is not a protocol.

The market's own selection statistic points the wrong way. `ScoreInput` (`rating/interface.go:15`) carries provider and model statistics, price per second, provider stake and minimum stake; the default scorer weights tokens per second, time to first token, session duration, success ratio and a stake term, then adjusts for price (`scorer_default.go:21-29`). Quality of output is not an argument to it. Dropping the floor weakly improves throughput, weakly improves time to first token, and strictly improves the price the provider can afford to post. The ranking function rewards substitution.

4.5 Committing to an admissible set

The obvious response, forbid routing, is both unenforceable and wrong. Routing is how a provider serves heterogeneous demand efficiently, and the network already contains it. The alternative extreme, publish the pool, deletes routers as a business, because the pool and the policy are the product. There is a middle construction and it is standard.

A provider registers, before the request, a commitment $C = (root, floor, class, k)$ against its bid:

- **root**: a Merkle root over salted leaves $H(manifest_i || s_i)$, one leaf per servable model. A manifest fixes weight digests or upstream model identity, version, context length, quantisation and serving configuration. The salt keeps an unopened leaf opaque.
- **floor**: a quality floor on a named public benchmark, claimed for every leaf in the set.
- **class**: the class of selection rule, not the rule. Cheapest-above-floor, lowest-latency-above-floor and learned-score are observably different; the parameters stay private.
- **k**: the size of the set, published, because it bounds every privacy claim that can be made about the scheme.

Each response carries an opening: the leaf, its Merkle path, and a signature over chain id, diamond address, `sessionId`, `requestId` and the response hash, which prevents a flattering opening being replayed or moved between requests. Verification is one hash chain and one `secp256k1` recovery, a few hundred bytes on a response already kilobytes wide.

What leaks. Openings accumulate into the multiset of served leaves and their frequencies, which is the realised policy over the prompts a given customer sent. k is the dial. At $k = 4$ under a stable rule the policy is public within a day; at $k = 60$ it remains a distribution, though hiding among sixty concedes that the floor is doing little work. Estimating the policy to accuracy ε costs on the order of k/ε^2 paid requests per prompt class, and re-salting each epoch restarts that bill. We state this rather than claiming the scheme is private, because a router operator is the party most exposed by it and that operator is us.

Type the bid, not the model. Routing is a property of a provider's serving path, but `Model.tags` (`IModelStorage.sol:22`) is written by the model owner. The node already reads

those tags to decide whether to demand an attestation from a provider’s backend (`isTeeModel`, `proxy_receiver.go:123`), which is a model-owner field gating a provider-side check. Put *fixed* against *routed* on the bid, where the provider owns it, and buyers filter on a field rather than on a convention.

4.6 Why disclosure needs verification

A commitment makes substitution decidable. It does not detect it. Nothing in Section 4.5 stops a provider from opening leaf i and serving something else, and nothing establishes that the floor or the rule class held. Disclosure without a way to check it is a statement, and the market already has statements.

Three questions get merged in discussions of this and should be separated.

Is the box genuine?

Morpheus answers this, and answers it well. That is the attestation stack of Section 3, with the two rented dependencies named there.

Did inference happen at all?

Nothing answers this.

Were these the advertised weights?

HideNSeek attempted it. Its last commit is 2024-07-29 and `main.py` is zero bytes on disk.

The proposal is three layers, cheapest first.

Commit. Add `bytes32 modelCommitment` to `Bid` and to `postModelBid`, emitted in `MarketplaceBidPosted`. One `SSTORE` per bid, none per request. A zero commitment stays legal and pledges nothing, which honestly describes every bid on the network today.

Bind. The 64 bytes of `TDX report_data` are fully allocated by the existing TLS binding, so the layout must not change. Change what the GPU nonce *is* instead:

$$\text{gpuNonce} = \text{SHA256}(\text{"mor-gpu-nonce-v1"} \parallel c \parallel \rho),$$

with c the model commitment and ρ a fresh random value published beside the quote. Freshness is preserved, the existing CPU-to-GPU nonce cross-check is untouched, and the Intel-signed quote now transitively covers c . Per response, the enclave’s TLS key signs (`sessionId`, i , c , $H(x)$, $H(y)$) for request index i , input x and output y , and the Merkle root over those leaves rides in the close-out receipt. Without a TEE the same statement degrades to a plain `secp256k1` signature, which is weaker evidence but still adjudicable.

Probe. Sample sessions with known-answer challenges and slash on contradiction. Let γ be what substitution saves as a fraction of epoch revenue, λ the fraction of stake slashed on a caught contradiction, and β the probability that a single probe catches a substituting provider. Expected penalty exceeds expected saving when

$$m \geq \frac{1}{\beta} \ln\left(\frac{\lambda}{\lambda - \gamma}\right)$$

probes per provider per epoch. At $\gamma = 0.9$, $\lambda = 1$ and $\beta = 0.3$ that is eight [illustrative]. Two features of the bound matter. It carries no traffic term, so the probe duty cycle falls as $1/\text{volume}$ and the scheme gets cheaper as the market grows. And it requires $\lambda > \gamma$: stake smaller than the prize deters nothing at any probe rate. That condition is already close to satisfiable here, because `_claimForProvider` denominates a period’s earnings in the same units as `provider.stake`. The bond exists; it is currently only a withdrawal delay.

4.7 Why verification wants an execution environment you specify

Per-audit verification is affordable anywhere. Per-request verification is a gas question, and gas is the one thing a tenant does not set.

Verifying a FIPS 204 ML-DSA signature costs 50,000 to 150,000 gas as a native precompile and upward of ten million in Solidity. The second figure does not merely cost more, it does not fit in a block anyone will pay for. The same holds for the lattice primitives that Section 5 argues receipts will eventually need. A rented execution environment will not add a precompile because one tenant’s market needs a predicate cheap.

Renting Base was the right call and we want to say that plainly. It bought a working market with settlement, staking, disputes and an upgrade path years before a chain of Morpheus’s own could have existed, and most protocols that build a chain first never reach the point where the chain matters. The costs are specific and worth naming rather than asserting in general:

- **A settlement rule enforced in blocks you do not produce.** `SIGNATURE_TTL` is ten minutes against a five-minute `MIN_SESSION_DURATION`, and `_rewardProviderAfterClose` withholds provider funds on a dispute only when the close is early (`SessionRouter.sol:279-289`). An inclusion delay comparable to a session length turns an early close into a late one and deletes the consumer’s remedy. Whether that has ever happened is a measurement, and Section 10 says who should take it.
- **A fee asset you do not issue**, so a consumer needs a second asset before opening a session and fee revenue leaves for another chain’s fee market.
- **A precompile set you cannot extend**, which is the binding one.

A sovereign chain changes four things. Protocol invariants become consensus transactions: “used stake cannot be recycled inside the same UTC day” becomes a validity condition every validator checks, rather than a branch spread across facets sharing a storage layout. MOR becomes the gas asset. Validator identity carries the session material, so address-to-key binding is read from state rather than asserted in a frame, which retires the parallel `secp256k1` PKI described in Section 5.3. And verification becomes a precompile, which is the one that Section 4.6 rests on.

Lux’s construction is three transactions against the upstream Lux P-chain and then independence: `CreateChainTx` creates the chain and its genesis, `AddValidatorTx` sets the genesis validator set, and `ConvertNetworkTx` promotes the network from inherited security to its own set under an on-chain staking contract (`platformvm/txs/convert_network_tx.go`). Afterwards the L1 runs its own primary network with `networkID` equal to its EVM `chainID`, one number for wallets, indexers and key derivation, and its validators stop talking to Lux at runtime. Sovereignty is not binary: a hybrid mode that restakes the parent while growing its own set pays the security budget as that set grows rather than on day one.

We are not recommending this today. Section 10 places it last and Section 11 names the condition under which it is a bad idea.

4.8 The chain, restated

Because	therefore
settlement pays for time, not tokens	model selection is the whole margin (LP-315)
selection is unobserved and unpriced	routing and substitution are one functional
they are indistinguishable on chain	a provider must declare an admissible set (LP-316)
a declaration is only a statement	the claim must be checkable (LP-313)
checking must be per request, not per audit	the predicate must be a precompile (LP-314)

Each step is implementable without the next. LP-315 needs no contract change at all. LP-316 needs a manifest field and a response field. LP-313 needs one facet replacement. LP-314 needs a chain. The chain is real but each link stands on its own, which is why the sequencing in Section 10 is not all-or-nothing.

5 Confidentiality

5.1 The asymmetry in the transport

Consumer prompts cross :3333 as plain JSON. Provider responses are ECIES encrypted. `SessionPromptRequest` takes the provider's public key as an argument and never uses it to encrypt the payload (`morrrpcmessage/mor_rpc.go:471`); `SessionPromptReq.Message` is a string, not a ciphertext (`request.go:55-60`); and all four `EncryptString` call sites in `proxy_receiver.go` run provider-to-consumer (:255, :335, :611, :654). The protected half is the half that usually matters less.

We do not think this was a decision. It has the shape of an asymmetry that appeared once and was never revisited, which is how most of them appear. It is also the cheapest thing in this document to fix and the one with the shortest argument in front of it.

5.2 What survives the obvious fix

Encrypting the request closes the immediate hole and leaves the structural one. Every confidentiality primitive in the tree is `secp256k1`: ECIES payloads in `lib/crypto.go`, morrrpc frame authentication, ECDSA approvals in `SessionRouter`. Worse, the ECIES key is the same wallet key handed to `bind.NewKeyedTransactorWithChainID` (`blockchainapi/service.go:1796`), loaded from `WALLET_PRIVATE_KEY` or the OS keychain (`config.go:48`). Its public value is therefore recoverable from any Base transaction the account has ever broadcast.

An adversary with a cryptographically relevant quantum computer does not attack recorded ciphertext under that arrangement. It lifts the static private key off public chain data and reads every session it captured. The asset in an inference market is the payload, and a payload does not become less sensitive with age, so the relevant horizon for this property is the working life of the person who typed the prompt rather than the life of a session.

Attestation does not reach this. The 5,439 lines establish what runs inside an enclave. They establish nothing about the channel that reaches the enclave, and a passive recorder on :3333 never touches the enclave.

Mechanism. Session establishment becomes hybrid: X25519 and ML-KEM-768 combined through HKDF, so the session key survives the failure of either primitive. Traffic keys derive only after the transcript is bound to the on-chain session identity, that is to chain id, diamond address, `sessionId`, `bidId`, provider and user, so a channel cannot be transplanted between sessions. Approvals become ECDSA $\wedge$ ML-DSA-65, with the lattice leg checked where the approval is already verified once, inside the consumer's proxy-router.

Base has no ML-DSA precompile and we will not pretend otherwise. Settlement stays classical until a chain underneath it has one, which is the honest version of the argument in Section 4.7: solvency has a short horizon and confidentiality an unbounded one, so they do not need the same primitive at the same time. Migration is suite-version negotiation rather than a fork, with the suite identifier inside the transcript hash so a downgrade fails as a bad signature rather than silently. Lux moved validator authentication onto a hybrid classical and ML-DSA suite on a live mainnet with no downtime, which is the only evidence we have that this shape survives contact with an operating network.

5.3 Peer identity is asserted, not resolved

Binding the transcript to the funded session closes a second thing, and this one is exploitable today without a quantum computer.

`proxy_receiver.go:526-528` stores, on a session request, the mapping `user:<addr>  $\rightarrow$  req.Key`. The signature on that request is verified with `req.Key` itself (`controller_morrrpc.go:125`), so the message asserts an identity and is checked against its own assertion. Any peer that can reach :3333 can therefore rebind the public key a provider associates with a victim's address. The consumer side has the same shape at `proxy_sender.go:288-295`, seventy lines below the on-chain resolution the same file already implements at :233.

The fix does not require the handshake. `VerifySignatureAddr` already exists (`mor_rpc.go:625`) and is used on the ping path. Anchoring the `session.request` signature check to `req.User`, and requiring `req.Key` to recover to that address, is a node-side diff with no wire change, no new dependency and no contract upgrade. A patched provider protects its own users whatever the client runs. Under the full handshake the peer key comes from the recovered signer and the `SessionOpened` event, and never from the message.

We will open this under the existing bounty with a proof of concept rather than publishing it further. Everything else in this section rests on that check.

5.4 A router that learns from traffic it cannot read

A learned router improves because it observes what happened after each choice. Section 5 closes the prompt inside a post-quantum session and Section 4.6 closes the inference inside an enclave. The learner and the confidentiality argument want opposite things from the same bytes, and the reconciliation is not rhetorical: it is a statement about where the feature extractor runs and what its output reveals.

A ledger row is (ϕ, m, r, c, ℓ) : a bucketed feature vector $\phi \in \mathbb{R}^d$, the model chosen, a scalar quality score, cost and latency, driving a policy toward $r - \lambda c - \mu \ell$. No prompt text, no completion text and no embedding appears in any field. The quality score comes from a judge model running inside the same enclave that terminated the session: it reads plaintext and emits one number. Consumer feedback, accept or retry or discard, is content-free by construction and is the stronger signal. Updates ship as aggregated ΔW over a minimum batch, which makes each shipped object a statement about traffic rather than about a request.

Extraction has to happen inside the confidentiality boundary or the property never existed. Compute ϕ outside the enclave and the prompt has already left. The extractor is therefore part of the attested workload and the ledger's only writer, pinned in the same golden set the verifier already fetches. That is not new machinery; it is the machinery of Section 3 pointed at a new binary.

What a feature vector still says. Content-free is not leak-free.

- ϕ carries task class, language, length bucket, difficulty estimate and tool markers. Anyone holding the ledger recovers a usage profile. Three short legal questions and a long code generation at 04:00 is a profile, and that is the price.
- Membership inference is the sharp risk and it scales with d . A 1024-dimensional embedding row fingerprints its prompt; thirty-two coarse buckets do not. Fix d small, quantise every field, bar embeddings at the schema level, and publish how many distinct prompts collide on a single row.
- Inversion against the policy recovers the feature regions where reward ran high, summed over traffic. It stops being aggregate the moment a policy row is fit to one account, so per-tenant policies stay in the enclave, gradients are clipped, and nothing ships below the batch floor.
- We do not claim differential privacy. An honest claim names a mechanism, a noise scale and an accountant, and a ledger that lives for years exhausts any fixed ϵ . DP on the update is a separate piece of work with a budget, not a word borrowed here.

Under the threat model of Section 5 the ledger is the long-lived half, so harvest-now applies to storage at least as much as to the wire. Rows are sealed under ML-KEM, signed under ML-DSA, and keyed by a rotating session pseudonym rather than by a consumer address. A feature vector written today is an artefact an adversary keeps.

5.5 Custody follows confidentiality

A Morpheus provider runs one `secp256k1` key for three jobs with three different lifetimes and three different blast radii: it holds stake, it prices bids, and it signs every `morrpc` frame and receipt on a TCP port that must be reachable from the open internet. A traffic key should be minutes old and disposable. A stake key should never sit on a host that accepts inbound connections. Once Section 5.3 makes the traffic key the thing an attacker on `:3333` is actually attacking, the conflation

stops being theoretical.

The indirection is built. `_isValidProviderReceipt` already accepts a delegate or an `owner()`, and `proxy-router` mirrors it: `OnChainProviderAuthResolver` calls `owner()` on the consumer's own RPC, keyed by the address the consumer chose, so it cannot be forged in transit.

The proposal is to finish it. The provider address becomes a custody contract whose `owner()` is a threshold key in a KMS, offline from every node, and stake authority lives there and nowhere else. Bid authority delegates separately, since `DELEGATION_RULES_MARKETPLACE` already gates `postModelBid` and `deleteModelBid` (`Marketplace.sol:68,119`), so a warm key that can reprice but cannot unstake is one delegation away.

Traffic authority is the right that does not exist yet, and its absence has a concrete cost. `_isValidProviderReceipt` accepts `DELEGATION_RULES_PROVIDER`, which is the same right that gates `providerRegister` and `providerDeregister` (`ProviderRegistry.sol:30,62`). Letting a datacenter sign receipts therefore also lets it deregister you. That is forced delisting rather than theft, because withdrawal still addresses the provider, but it is the wrong radius for a key on a box with `:3333` open. Split it and rotation stops meaning re-registration: rotating or revoking a traffic key becomes a delegation write, leaving address, endpoint, bids and stake untouched, and consumers pick up the new signer on their next resolve.

Custody keys should be t -of- n rather than a file. Lux's threshold signing rotates holders without changing the public key, over two kernels: a `secp256k1` kernel, which is what Base requires since `ECDSA.recover` is the diamond's only verifier, and Pulsar, a lattice kernel whose combined output is byte-equal to a single-party FIPS 204 ML-DSA-65 signature. Pulsar is review-ready and not production-certified, and it does not verify on Base. It is the path for custody that must outlive `secp256k1`, not a claim about today. The `secp256k1` kernel buys the property that matters now: no machine holds a whole key.

An unrelated defect in the same path. `MyProvider` is the entire supply-side onboarding flow, and its port check reads a `TypeError` from `fetch` as proof that the port is open (`apiService.ts:470-477`). A `fetch` to a closed port fails in exactly that way, and `:3333` speaks `morrpc` rather than HTTP, so no browser can perform this check at all. Operators stake MOR to advertise endpoints that nothing can reach. The fix is a server-side probe completing one `morrpc` handshake, and it is a day of work.

6 Two repairs available with deployed primitives

Neither item in this section needs a new primitive, a new chain, or a cryptographic argument. Both are visible in code that shipped.

6.1 Delegated sessions

The hosted gateway multiplexes arbitrarily many Web2 callers onto sessions funded by one consumer wallet. The engineering around that pool is careful: idle-session claiming under `FOR UPDATE SKIP LOCKED`, a cheapest-bid walk with per-provider cool-off, spend fuses that ship inert and arm per environment, day-locks derived from close receipts. Better fuses do not reach what is left. One balance is a liveness dependency for every caller simultaneously; per-caller accounting lives in a credit ledger the chain cannot correct, while the routed session table carries no user id; and no caller has any claim on the session serving them. The address that staked, that gets refunded, and that `getUserSessions` returns is the gateway's.

Nothing on chain has to change for this to stop. `openSession(address user_, ...)` validates the caller against `user_` under `DELEGATION_RULES_SESSION` at `SessionRouter.sol:90` and then pulls stake from `user_` at `:104`. Caller and owner are already separate variables, and `SessionRouter.test.ts:208` already tests opening a session from a delegatee address. One argument in the Go node forecloses the whole construction:

```
// registries/session_router.go:61 and :78
sessionTx, err := g.sessionRouter.OpenSession(opts, opts.From, stake,
                                              directPayment, approval, approvalSig)
```

`opts.From` is the node’s own address, fed by `OpenSessionByBidId` reading `s.GetMyAddress(ctx)` (`blockchainapi/service.go:1259`). The session owner is hard-wired to the caller.

Under delegation, the gateway holds a right instead of custody. The consumer writes `delegateContract(gateway, diamond, DELEGATION_RULES_SESSION, true)` to the registry directly; the gateway opens with `user_` set to that consumer. Stake leaves the consumer’s balance and every exit is addressed back to them, `_rewardUserAfterClose` to `session.user` and `withdrawUserStakes` to `user_`. That is the loss bound: pooled, a compromised gateway key yields the pool; delegated, the worst it does is burn a stipend claim and day-lock that one consumer’s principal for under a day, because the contract offers it no other address.

Two caps already hold in deployed code and revoke without the gateway’s cooperation: the consumer’s ERC-20 allowance to the diamond, and `maxSessionDuration`. What a boolean `checkDelegateForContract` conveys should be said plainly, which is nothing else: no per-open size limit, no expiry, no model restriction. The registry holds the first of those already, since `delegateERC20(to, MOR, rights, amount)` publishes an upper bound readable through `checkDelegateForERC20`. Expiry and model scope want a small facet, and that is the second move, not the first.

The key problem, head on. A delegation needs a consumer key, and account-native users do not have one. That cost does not disappear. But the gateway already knows which callers do: `wallet_links` stores addresses proved by SIWE, and that cohort moves with no new onboarding at all. For everyone else the pool stays, much smaller, with a sponsored delegation from a smart account as the route off it.

6.2 Anchoring the compute domain to the capital domain

Capital lives on Ethereum. Compute lives on Base. The Base diamond holds its own copy of the emission schedule, a `Pool[]` array written by `setPoolConfig` under `onlyOwner`, whose `@dev` comment says the parameters “should be the same as in Ethereum L1 Distribution contract” at `0x47176B2Af9885dC6C4575d4eFd63895f7Aaa4790`, and instructs the reader to call `Distribution.pools(3)` to check (`SessionRouter.sol:58-60`). Nothing does. MOR crosses as a LayerZero OFT and a single Python service sits between the two economies.

The coupling variable is `getComputeBalance`, cumulative compute emission minus `providersTotalClaimed`. Through `stakeToStipend` it determines how much session time a given stake purchases, which makes it the only channel by which realised demand feeds back into what the market can sell. Ethereum cannot read it.

Both domains call the same deployed `LinearDistributionIntervalDecrease` with independently configured arguments. A relative error β in the decrease rate d produces a relative error of

$$\frac{\beta dn}{R_0 - dn}$$

in the remaining pool at day n , where R_0 is the initial reward. This diverges as $R_0 - dn \rightarrow 0$: the mirror is least reliable exactly when the pool is smallest and the numbers matter most.

What must cross, and what must not. Minimising the crossed state is the design, so it should be bounded explicitly. Four scalars and two commitments per UTC day: `providersTotalClaimed`, settled session-seconds, sessions closed, and `min(balance, allowance)` of the funding account, which is the real cap on payable capacity; plus a Merkle root over per-provider claim totals and a digest of the pool config in force. Excluded: sessions, bids, prices, prompts, model identity, telemetry. A daily digest of fixed size is the right object. A message bus is not.

Three levels of verification, with the adversary named. The adversary is the diamond owner together with the funding-account operator, who can rewrite the pool config, repoint `fundingAccount`, or simply stop funding, all silently.

Attested digest with a challenge window.

Nearly free. Guarantees only that a false digest cannot pass unopposed.

Storage proof against a finalised Base output root.

Removes the trusted party, using machinery Ethereum tooling already ships, at the price of the rollup's fault window.

A native quorum certificate.

Removes that window and requires a compute domain that produces one. Lux cross-chain finality applies between Lux chains, not into a Base diamond. If this is the goal, it is Section 4.7's problem, not this section's.

The cheap intermediate. Nothing above needs Lux, and the first step needs nothing from anyone outside Morpheus. Three changes to Base and Ethereum as they stand: a `view` returning `keccak256` of the pool config each side holds, so anyone can diff them; `SessionClosed` (`ISessionRouter.sol:10`) widened to carry duration and settled amount, so demand is reconstructible from logs alone; and balance and allowance exposed alongside the daily budget. The first of those is an afternoon. Either the mirror holds, or the divergence is a number.

This matters because MRC authorises the compute allocation, and the figure that would tell MRC whether that allocation tracks real demand is `providersTotalClaimed`, sitting in Base storage MRC cannot see. Anchoring does not fix a governance process. It gives it a number to check.

7 Sessions as an agent substrate

This section is the one place where the programme is about new revenue rather than repair.

Per-second billing is a poor fit for chat, where the session is mostly a human reading. It is close to the right fit for an agent trajectory, where the gaps are the agent working. Model a trajectory as alternating phases $(g_1, t_1, \dots, g_n, t_n)$, with g_i seconds generating at $\bar{r}$ tokens per second and t_i seconds blocked on a tool. Write $G = \sum g_i$, $T = G + \sum t_i$ and $\rho = G/T$. A per-token price κ charges $\kappa \bar{r} G$ and ignores every t_i . A per-second bid p charges pT , an effective $p/(\bar{r}\rho)$ per token. They agree at

$$\rho^* = \frac{p}{\kappa \bar{r}}.$$

Chat sits far below ρ^* . An agent bursting against local tools sits near $\rho = 1$. Per-second is worse exactly where ρ collapses: forty seconds blocked on a slow external API buys forty seconds of GPU price and no work. The unit is right for one workload and wrong for another, and knowing which is which is a measurement nobody has published.

It is an admission problem before it is a pricing problem, and admission is factored well already. `HasCapacity(modelID)` is the entire interface; `simple` counts open sessions, `idle_timeout` counts sessions active inside a hardcoded fifteen minutes (`capacity_manager.go:72`). That is the right instinct at the wrong resolution. `PromptActivity` already carries `SessionID`, `StartTime` and `EndTime` (`storages/structs.go:32`), which is enough to seat a second consumer in the slot a parked one leaves idle. It is one more case in `CreateCapacityManager` (:112).

Two things a trajectory needs that a session is not.

Tools run on the provider. `CallAgentTool(ctx, sessionID, agentID, toolName, input)` (`ai_engine.go:195`) runs a registered agent's configured command as an MCP stdio child of the provider process. The provider executes the tool on its own credentials, is paid per second for as long as it takes, and the consumer authorises this by holding a session without seeing what ran. A provider running slow tools is paid more. Admission and settlement are the same question here, which is why the capacity work is a precondition rather than an optimisation.

A session spans neither a second bid nor its own funding. Conversation history keys on `chat_id`, a client header randomised when absent (`controller_http.go:217`), while tools and payment key on `sessionId`. A trajectory that wants two models therefore holds two sessions whose tool state cannot join even though their `chat_id` does. And `endsAt` is fixed at open with no extend, so outrunning the funding means resuming under a new `sessionId` with history intact and tool handles gone.

One client-side object closes both: a `chat_id` plus a set of live `sessionIds`, opened before the current one expires rather than after, one per bid, presenting a single tool namespace across them. Because latency under a per-second bid *is* cost, the policy falls out of the arithmetic: hold the cheap session open across waits, and spend into the expensive one only while generating.

The current agent story on the network is seven repositories. Five are Vercel-derived templates that set an OpenAI base URL at `api.mor.org` and never touch a session; `sim-studio-morpheus` and `Perplexica-morpheus` carry the name and little else. The marketplace bills in the unit agents consume and has no agent runtime. That is an opportunity rather than an indictment.

8 What we bring, named to repositories

Each fix in this document is a repository that exists today, not a plan. The mapping is direct.

Morpheus problem	Fix	Repository
Model selection is the margin	Learned router, per request	Enso (<code>api.hanzo.ai</code>) [verified]
No proof inference happened	Attested backend	<code>api.hanzo.ai</code> [verified]
Settlement is fakeable	Un-fakeable receipt	<code>mord</code> [verified]
No attestation root you control	Quorum + <code>SlashProvider</code>	<code>lux/ai</code> [verified]
No slashing, no timelock	Governance, Safe, timelock	<code>lux/dao</code> [verified]
DEX depth is protocol-owned	Order-book DEX “LX”	<code>lux/dex</code> [verified]
No compliant capital path	Securities exchange + ICO	<code>lux/exchange</code> [pending]
MOR has no yield use	Self-repaying lending vs MOR	<code>lux/liquid</code> [verified]
No fiat on-ramp	Accounts and cards	<code>lux/bank</code> [verified]
Renting a chain caps verification	L1 in minutes	<code>12-template</code> [verified]

`mord` is built. Forty tests pass as of 2026-08-29 [verified]. Faked work earns zero, and a substituted model is slashed, because the receipt commits to the model and the token counts the current contract discards. `lux/ai` runs attestation, a validator quorum, and `SlashProvider` against local `nvtrust`, SPDM and RIM, with no cloud dependency, which is the rented root named in Section 3. `lux/liquid` is a VaultV2 with a dozen adapters. `12-template` is how Hanzo and Zoo were each brought up; its post-quantum root is Quasar Horizon finality. `lux/exchange` is marked [pending] because it depends on SEC approval of a proposed regulation, not on engineering.

Of the post-quantum primitives, ML-KEM and FHE are activated on Lux mainnet [verified]; ML-DSA and SLH-DSA are present but not yet activated [verified]. We state this because Section 5 argues receipts will eventually need the lattice signatures, and they are not live today.

Demand. The fixes make the market honest. Demand makes it matter. Hanzo and Zoo ship consumer surfaces with a combined launch reach above 100,000,000 [client]: `hanzo.chat`, `hanzo.app`, `hanzo/dev`, `hanzo/desktop`, `zoo/app`, `lux.exchange`, `lux/wallet`. These are OpenAI-compatible clients. They can route into a decentralized inference tier without their users holding a wallet, through the same hosted-path shape Morpheus already built at `api.mor.org`.

9 The outcome

9.1 The funnel

Start from reach and end at verified sessions a day. The reach is ours [client]. Every step after it is modeled, and the assumption is named beside it.

Stage	Count	Assumption
Launch reach	100,000,000+	Hanzo + Zoo surfaces [client]
Monthly active	~2,000,000	2% of reach activates [illustrative]
Opt into private tier	~40,000	2% of active opt in [illustrative]
Routed at least once	~20,000	half of opt-ins route [illustrative]
Verified sessions/day	1,000–10,000	supply-bounded at 18 providers [illustrative]

The near-term ceiling is supply, not demand. Eighteen providers cannot serve more than on the order of 10^4 verified sessions a day [illustrative]. That is the point worth holding: the low case, roughly 10,000 sessions a day, already exceeds all currently published Morpheus demand, which is ~ 0 [verified]. The binding constraint moves from “is there any usage” to “is there enough supply,” which is a problem a network wants to have.

9.2 What the usage is worth

State the mechanism plainly. A fee on verified volume feeds the existing buyback. Revenue then scales with inference sold rather than with deposits.

Today the buyback runs on deposit yield, about \$511K over the trailing year, and that figure rises and falls with interest rates, not with usage [verified]. Put a fee on verified sessions instead. Assume a fee equivalent to \$0.05 per verified session [illustrative]. The low case, 10,000 sessions a day, is 3.65M sessions a year, about \$183K [illustrative]; the high case at the same fee is about \$1.83M [illustrative].

The size is not the argument. The derivative is. Yield revenue falls as rates fall. Usage revenue rises as sessions rise. A fee on verified volume moves the buyback onto the axis the emission is meant to reward, and it makes the receipt in `mord` the thing revenue depends on, which is why the receipt has to be un-fakeable before the fee means anything.

9.3 Sequence, and why us

Demand and verifiability first: routing that produces real sessions, and a receipt that settles them honestly. Token rails second: governance with a timelock, the LX order book, `lux/liquid`, `lux/bank`. An L2 and a raise third. A raise clears only into demonstrated on-chain revenue and a KYC base already onboarded through Hanzo IAM and `lux/wallet`, not into a projection.

Relevance. Morpheus needs two things it cannot build from inside its own tree: buyers, and an un-fakeable receipt. Hanzo has the buyers, a launch reach above 10^8 [client], and the router. Lux has the settlement primitive, `mord`, built [verified], and the chain that can price its verification in gas. No other counterparty brings both the demand and the proof to the same market. That is the case for this specific fix, made to this specific network.

10 Sequencing

10.1 What blocks what

Three of the ten dockets have no prerequisites and produce numbers the network does not currently have. Four depend on exactly one other. Two are a year of work. One is a decision rather than a task.

Docket	Subject	Blocked by	Contract change
LP-315	Routing alpha, measured	nothing	none
LP-317	Session agents, ρ measured	nothing	none
LP-321	Config digest, first step	nothing	view function
LP-312	Transport, rebinding fix	nothing	none
LP-320	Delegated sessions	nothing	one read
LP-318	Provider custody	LP-312 in part	one constant
LP-316	Declared routing	LP-315	manifest first
LP-313	Verifiable inference	LP-316	one facet
LP-319	Confidential routing	LP-312, LP-313	none
LP-314	Sovereign L1	LP-313	a chain

The one dependency worth stating in prose is LP-316 on LP-315. A disclosure regime nobody has a reason to comply with is a form. If routed providers are earning materially more than pinned ones, disclosure becomes something a provider wants to sell against competitors who disclose nothing. Measure first, then propose the rule.

10.2 What could ship in a quarter

Five items, none of which requires a 5-of-9 signature.

Work Hanzo does, and publishes whatever else happens

- Register a provider on the Base diamond, bid on an existing `modelId`, serve it from our endpoint with the router choosing the backend, and publish α measured against a pinned control over the same traffic. Bidding is permissionless inside the price band, so this needs no facet upgrade and no cooperation.
- Write the third capacity manager against `dev` and publish ρ , generating seconds against paid seconds, over that provider's own traffic. Both figures are absent from all 62 repositories, and both are the network's whether or not anything else here is adopted.
- Thread an optional session owner through `OpenSessionByBidId` into `session_router.go`, defaulting to `opts.From` so existing deployments behave identically. The contract tests already exist.
- Publish the routing ledger schema with d fixed and the extractor as a reproducible build, and submit its measurement for pinning in the golden set the verifier already trusts.

Work a Morpheus contributor does, mostly in an afternoon each

- Land the `session.request` signature anchor: check against `req.User`, require `req.Key` to recover to it. One node-side diff. We will file it through the bounty with a PoC first.
- Add the pool-config digest view to the diamond, read `Distribution.pools(3)` on Ethereum, and compare. No counterparty, no upgrade beyond a facet.
- Export `SessionOpened` and `SessionClosed` and join them against Base inclusion history to compute the fraction of closes pushed past `endsAt`. That single number decides whether the liveness argument in Section 4.7 is real or theoretical, and it is a day of work.
- Replace the `MyProvider` port check with a server-side morrpc handshake probe. This is unglamorous and it is currently the first thing every new provider encounters.

10.3 What is a year

LP-313 is a year in the sense that its three layers are three separate pieces of work with different owners. The commitment field is a facet replacement and a week. The GPU nonce binding is a change to an attested workload plus a verifier update, which means a release cycle and an audit. The probe and slashing layer is a new economic mechanism with a governance dependency, and mechanisms with governance dependencies do not ship in a quarter on any network.

LP-314 is a year and should not start until LP-313's roadmap is real. A chain is justified by a predicate that must run per request; without that predicate the case reduces to fee capture, and

fee capture does not close until roughly 10^5 sessions per day. We say that as the party that would benefit from Morpheus running on Lux.

LP-318's custody contract and KMS adapter are a quarter of our work and a one-constant change of yours, but the useful version of it waits for LP-312's transport, because splitting traffic authority matters most once the traffic key is the thing being attacked.

10.4 Staffing, realistically

Native output on this org is roughly 100 to 500 commits a month from about five people, and some of what looks like native work is agent-written. That is the constraint every proposal here has to survive.

What it implies: the Morpheus-side work in Section 10 totals about four engineer-weeks, deliberately, and none of it is on the critical path of anything else the team is doing. The larger items are ours to staff. Where a proposal needs a facet replacement we have said so explicitly and kept it to one transaction under the existing Safe, because a five-person team's scarcest resource is not engineering time, it is the number of upgrade transactions it can review properly in a quarter.

We would rather land four small things that are obviously correct than one large thing that needs a governance vote.

11 Limits

11.1 What no one in this programme can fix

Nothing here proves that the arithmetic of an inference was right. An enclave that loads the pledged weights and then truncates the attention window produces a valid receipt under every mechanism in Section 4.6. Proving inference in zero knowledge is out of reach at these model sizes and we are not going to imply otherwise.

Nothing here constrains what a provider does with plaintext it necessarily holds. A sound channel delivers the prompt to a backend the provider configured. Attestation narrows this to what a measured binary does, which is narrower than trust and wider than nothing.

Nothing here creates demand. A better-verified market with no users is a better-verified empty market, and every mechanism in this document is a cost until volume exists to amortise it.

A compromised diamond owner can rewrite state before a digest is taken, so Section 6.2 transports the compute domain's accounting faithfully without making it true.

And per-second pricing, which is the premise of the whole first chain, is not something we propose to change. Everything in Section 4 is downstream of a decision we think was correct.

11.2 What depends on data the network does not publish

Three quantities in this document are assumptions and are marked as such wherever they appear.

α is unmeasured. The illustrative figures in Section 4 are shape, not measurement, because no price, volume or session-length series exists in any of the 62 repositories. If the real spread between admissible backends is small, LP-315 is a smaller idea than it looks and LP-316 loses most of its motivation.

ρ is unmeasured. If agent traffic on this network turns out to be tool-blocked most of the time, per-second pricing is worse for agents than the argument in Section 7 implies, and the honest conclusion would be that sessions want a suspend primitive rather than that agents want sessions.

The fraction of closes pushed past `endsAt` is unmeasured, and it is the whole empirical content of the liveness argument against renting Base. If that fraction is zero over a year of history, the strongest concrete cost of tenancy disappears and the case for LP-314 rests on precompiles alone.

We have proposed the measurements rather than the conclusions in all three cases, and where a first step is ours we will publish the number whether or not it supports the proposal attached to it.

11.3 Where our interests and yours diverge

Four places, and naming them is the point of this section.

We benefit from routing being legitimate. Hanzo operates a learned router. Sections 4 and 4.5 argue for a regime in which routing is disclosed and permitted rather than prohibited, which is the regime in which our product is sellable on this network. A network that decided instead to require pinned models and verify them would be a worse fit for us and might be better for consumers. We think the disclosure construction is the right answer on the merits, and you should discount our confidence accordingly. The partial defence is that LP-316 constrains us more than anyone: k , a floor and a rule class are things we would have to publish, and no competing router publishes them today.

We benefit from Morpheus running on Lux. Section 4.7 is a proposal to move settlement onto infrastructure we operate. We have tried to write it as a decision with thresholds rather than a recommendation, we have put it last in the sequence, we have said that fee capture alone does not justify it, and we have named a team that cannot staff chain operations as a veto rather than an obstacle. It is still the item in this programme where our interest is most obviously served, and it is the one to read most sceptically.

We want traffic diversity, and that is what LP-319 buys us. A content-free ledger fed by a live per-second market is training signal we cannot generate from our own endpoint. Section 5.4 is written to make the leak surface auditable precisely because we are asking for something. The schema, the field list and the extractor build are the concessions; they exist because we are the beneficiary.

We are a competitor in one line of business. Hanzo sells inference through an OpenAI-compatible endpoint. So does `api.mor.org`. If this programme succeeds, the hosted Morpheus gateway and our endpoint compete for some of the same buyers, and we would be selling into a market whose verification and settlement layer we helped design. We do not think that is disqualifying, but it is a reason to insist that every mechanism here be specified well enough for someone else to implement without us. Each one-pager says so in the same words: implementable without adopting Lux or Hanzo software. That sentence is a commitment, and it is the one we would most like to be held to.

12 Docket index

Docket	Title	Section
LP-312	Post-Quantum Session Confidentiality	5
LP-313	Verifiable Inference	4.6
LP-314	A Sovereign Layer 1 for Inference Settlement	4.7
LP-315	Routing Alpha	4
LP-316	Declaring a Router	4.5
LP-317	Sessions as the Agent Substrate	7
LP-318	Provider Identity Without Provider Keys	5.5
LP-319	Learning From Traffic You Cannot Read	5.4
LP-320	Delegated Sessions	6.1
LP-321	Anchoring Marketplace State	6.2

Every code citation in this document was checked against the working trees of the `MorpheusAIs` repositories in August 2026. Line numbers drift between releases; identifiers do not, and we have preferred identifiers wherever one was available. Corrections, and in particular measurements that contradict the assumptions in Section 11, are the most useful reply we could receive.